STL Fundamentals

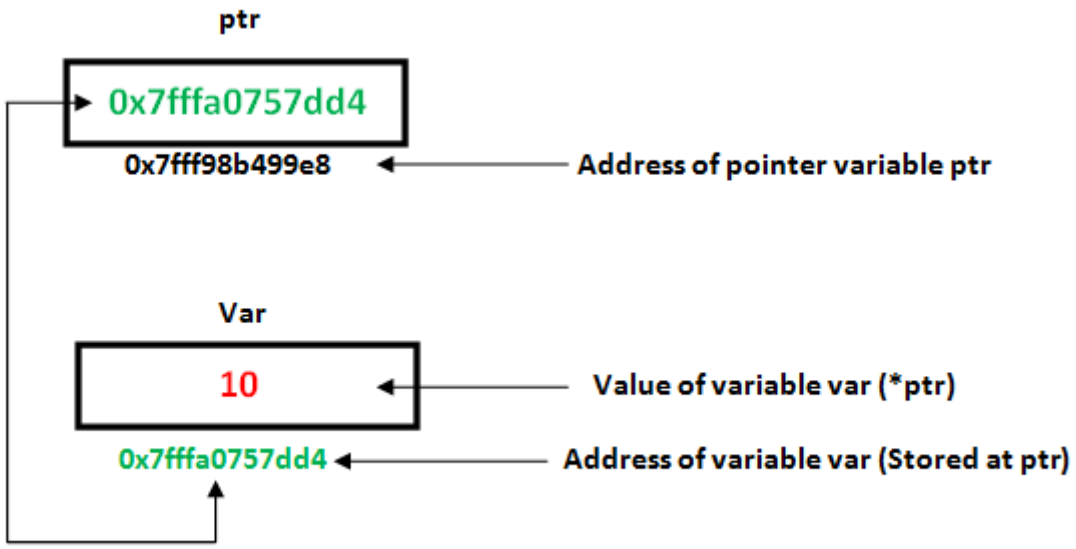
Core Concepts for mastering STL

> Pointers
>
> ## Pointers
>
> Your mental image while dealing with anything related to memory should be that of a box.
>
> Every variable, arrays, everything that you create in a program, will have a name that is local to the program and an address of where it is stored in the RAM.
>
> In C++, a pointer is a variable that stores memory address of another variable. Pointers are one of the most powerful features of C++ because they allow for direct memory access and manipulation, which leads to efficient and flexible code.
>
> ## Key Concepts of Pointers
>
> 1. **Pointer Declaration**
> To declare a pointer, use the `*` symbol.
>
>  ```
>  int* ptr;
>  ```
>
> 2. **Address-of Operator (`&`)**
> The `&` operator is used to get the memory address of a variable.
>
>  ```
>  int x = 10;
>  int* ptr = &x;
>  ```
>
> Here, `ptr` stores the address of variable `x`
> 3. **Dereferencing Operator (`*`)**
> The `*` operator is also used to access the value stored at the memory address a pointer holds (dereferencing the pointer).
>
>  ```
>  int y = *ptr;
>  ```
>
> This assigned the value pointed to by `ptr` (which is the value of `x`) to `y`.
> This is also known as **Value At** operator.
> 4. **Null Pointer**
> A pointer that doesn't point of any memory location is called a null pointer.
> It is usually initialized with `nullptr`
>
>  ```
>  int* ptr = nullptr;
>  ```

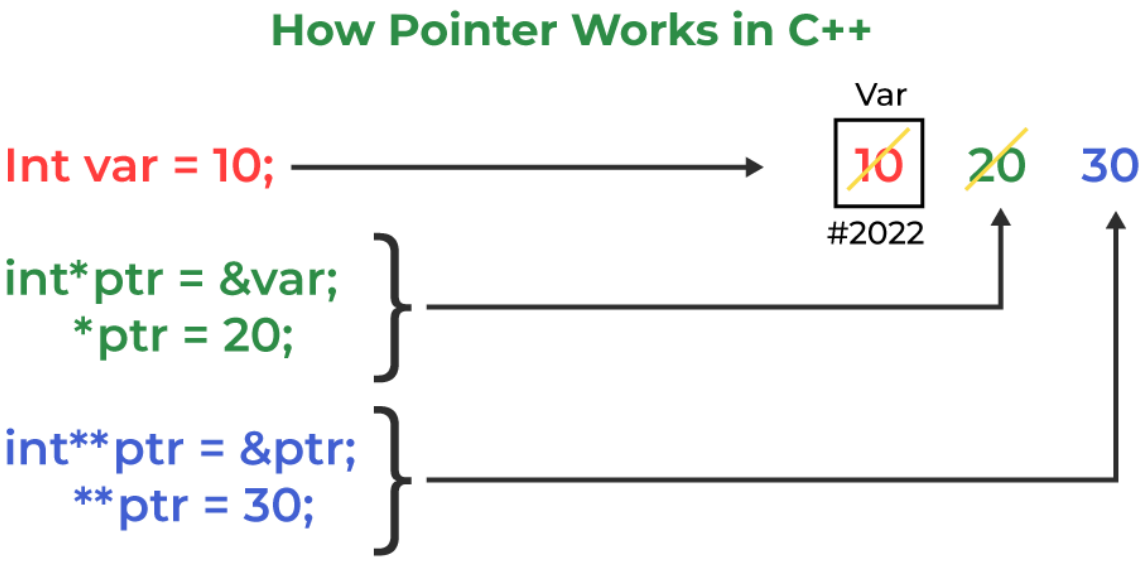
5. **Pointer Arithmetic**
   Pointers support arithmetic operations like addition or subtraction, which allows moving the pointer across memory locations, especially in arrays.

```
int arr[] = {1, 2, 3};
int* ptr = arr;    // points to arr[0]
ptr++;    // now points to arr[1]
```

Visual Representation of Pointer



Working of Pointers



Here, `**ptr` is a pointer to `int *` & it's known as **pointer to pointer**.
So, `**ptr` stores address of another pointer variable of type `int *`.

Array Pointers

Array Pointers

Array pointers, also known as **pointers to arrays**, are pointers that point to the memory location of an array. Understanding how arrays and pointers interact in C++ is essential for handling arrays efficiently, especially when dealing with large amounts of data.

Key Concepts of Array Pointers

1. **Array to Pointer Relationship**
   In C++, the name of an array acts as a pointer to the first element of the array.
   This means that if you have an array say `arr`, then the name `arr` is essentially a constant pointer to the 1st element (`&arr[0]`).

   ```
   int arr[5] = {10, 20, 30, 40, 50};
   int* ptr = arr;    // ptr points to arr[0]
   ```

2. **Accessing Array Elements with Pointers**
   You can access elements of array using Pointer Arithmetic. Since the array name is a pointer to the 1st element, you can increment or decrement the pointer to move through the array.

   ```
   int arr[3] = {1, 2, 3};
   int* ptr = arr;

   cout << *ptr << endl;    // Output: 1 (arr[0])
   cout << *(ptr + 1) << endl;    // Output: 2 (arr[1])
   cout << *(ptr + 2) << endl;    // Output: 3 (arr[2])
   ```

3. **Pointer to an Array**
   A pointer could also point to the entire array rather than just a single element. To declare a pointer to an array, you need to specify the size of the array.

   ```
   int arr[5] = {10, 20, 30, 40, 50};
   int (*ptr)[5] = &arr;    // ptr is a pointer to the entire array
   ```

   Here, `ptr` is a pointer to an array of 5 integers. To access elements of the array using this pointer, you can dereference it.

   ```
   cout << (*ptr)[0] << endl;    // Output: 10
   cout << (*ptr)[1] << endl;    // Output: 20
   ```

Difference between Array Pointer & Pointer to Array

- **Array Pointer (Pointer to the 1st element of an array)** :
  Declaring `int* ptr = arr;` means `ptr` points to the 1st element of the array.
  Pointer Arithmetic (`ptr + 1`, `ptr + 2`, etc.) will allow you to access subsequent elements.
- **Pointer to the entire array** :
  Declaring `int (*ptr)[5] = &arr;` means `ptr` points to the whole array, and dereferencing

`(*ptr)` gives you access to the array. Here, Pointer Arithmetic will move the pointer across entire arrays (if there were more arrays).

Mechanics of Array Pointers in C++

```
int arr[] = {2, 3, 5, 7, 11};
cout << *arr << '\n';    // Prints 2
cout << *(arr + 1) << '\n';    // Prints 3
cout << 2[arr] << '\n';    // Prints 5
```

Behind the scenes, C++ also decodes `arr[i]` as `*(arr + i)`

Note : C++ also decodes `i[arr]` as `*(i + arr)`
Hence, `arr[i]` ≡ `i[arr]` ≡ `*(arr + i)` which is fairly weird

References

References

In C++, **References** provide an alternative name or alias for existing variable.
A reference is not a separate object, but another name for the same memory location.

References are often used in function parameters to allow functions to modify the arguments passed to them.

Key Points about References

1. **Declaration** : A reference is declared using the `&` symbol.

```
int a = 10;
int &ref = a;   // 'ref' is a reference to 'a'
```

2. **No Re-Assignment** : Once a reference is assigned to a variable, it cannot be changed to refer to another variable.

```
int b = 20;
ref = b;    // This does'nt change reference; instead, it assigns value of
'b' to 'a'
```

3. **Must be Initialised** : References must be initialised at the time of declaration. You can't create an uninitialised reference.

```
(int &) ref;    // Invalid, must initialize a reference
```

4. **Usage in Functions** : One of the main uses of references is in function parameters, allowing functions to modify the arguments passed to them without copying them.

```
void modify(int &x) {
    x = 100;
}

int num = 10;
modify(num);    // 'num' becomes 100 because 'x' is a reference to 'num'
```

5. **Const References** : A reference can be declared as `const`, meaning that it cannot be used to modify the referenced variable.

```
void print(const int &x) {
    cout << x;
}
```

NOTE :
`&` on LHS of `=` is actually for **References**
`&` on RHS of `=` is actually **Address-of** operator in **Pointers**

Difference from Pointers

- **References** must always be initialised, while **pointers** can be `nullptr`.
- **References** cannot be reassigned to refer to another object, while **pointers** can be changed to point to different objects.
- Accessing a **reference** is syntactically similar to accessing the variable itself, while **pointers** require dereferencing (`*`).

What is STL ?

The Standard Template Library (STL) is a powerful library that provides a set of common classes and functions, including algorithms and data structures like vectors, lists, stacks, queues and more. It is designed to handle generic programming, allowing developers to write code that works with any data type.

Key components of STL

1. Algorithms
   STL provides wide range of algorithms to perform common operations on data, such as searching, sorting, and manipulating data. These algorithms are implemented as template functions, meaning they can work with any container that supports iterators.

   - rand()
     ### rand()
     In C++, `rand()` function is a part of the C standard library (`cstdlib`) and is used to generate **pseudo-random numbers**. While not technically part of the STL, it's commonly used in conjunction with STL containers & algorithms.

Key Points about `rand()`

1. **Function Definition** : The `rand()` function returns a pseudo-random integers in the range `[0, RAND_MAX]`, where value of `RAND_MAX` is library-dependent but is guaranteed to be at least 32767 on any standard library implementation.

   ```
   int rand();
   ```

2. **Seeding with `srand()`** : By default, the random number generator is seeded with the same value every time you run the program, which leads to the same sequence of random numbers.

   To generate different sequence of numbers, you can seed the generator using the `srand()` function.

   ```
   void srand(unsigned int seed);
   ```

   The common way to seed the random number generator is by using the current time, which ensures a different sequence of random numbers each time the program is run.

   ```
   #include <ctime>

   srand(time(0));    // Seed with current time
   rand();    // now everytime your code runs, 'rand()' gives you a new value
   ```

   `srand()` function is used to change the default seed in the `random` library.

NOTE :
In Python, `rand()` generates floating point numbers
In C++, `rand()` generates integers

Generating random numbers in a Range

Write a function that generates random number in range `[a, b]`

```
int random_ab(int &a, int &b) {
        return a + (ran() % (b - a + 1));
}
```

Limitations of `rand()`

- **Quality of Randomness** : `rand()` is a pseudo-random number generator (PRNG) and not suitable for cryptographic purposes or high quality randomness. It generates numbers based on an algorithm that produces a deterministic sequence, which can be repeated if seeded with the same value.
- **Limited Range** : The maximum value that `rand()` can generate is limited by `RAND_MAX`. For generating random numbers in a much larger range, `rand()` might not be sufficient.

Modern Alternative `std::random` in C++11 and Later

C++11 Introduced a new random number generator framework in the `<random>` header, which provides better control and more advance random number generation mechanisms (e.g., different distributions and engines). This approach is more flexible and offers better randomness quality.

```
#include <random>
using namespace std;

int main() {
        // Create a random number generator
        random_device rd;    // Seed
        mt19937 gen(rd());    // Mersenne Twister engine
        uniform_int_distribution<> distr(1, 10);    // Range [1, 10]

        // Generate a random number in the range [1, 10]
        cout << "Random number between 1 and 10: " << distr(gen) <<
endl;

        return 0;
}
```

- random_shuffle() and shuffle()

random_shuffle()

The `random_shuffle()` function in C++ was used to randomly rearrange the elements in a given range,, producing a random permutation of the elements. It was part of the `<algorithm>` header in the C++ Standard Library.

Function Prototype

```
void random_shuffle(RandomIt first, RandomIt last);
void random_shuffle(RandomIt first, RandomIt last, RandomFunc rand);
```

- *first* : Iterator pointing to the beginning of the range
- *last* : Iterator pointing to the end of the range (the position one past the last element)
- *rand* (optional) : A function or callable object that provides random values

`random_shuffle` rearranged the elements randomly using an internal random number generator unless a custom generator was provided.

Example

```
#include <algorithm>

int main() {
        vector<int> v = {1, 2, 3, 4, 5};

        // Shuffle the vector using random_shuffle (deprecated in C++17)
        random_shuffle(v.begin(), v.end());

        return 0;
}
```

NOTE
However, it's important to note that `random_shuffle` was deprecated in C++14 and removed in C++17. In modern C++ (C++11 and later), the preferred function is `shuffle`, which is more flexible and requires and explicit random number generator.

Why `random_shuffle` was Deprecated and Removed ?

- `random_shuffle` used the `rand()` function internally, which was not a high-quality random number generator
- `rand()` has poor randomness and was implementation-dependent, which made it unsuitable for many modern applications that require good randomisation

Modern Replacement : `shuffle()`

Instead of `random_shuffle`, you should use `shuffle`, which allows for better control over the randomness by requiring an explicit random number generator (like one from the `<random>` library).

Function Prototype

```
template<class RandomIt, class URBG>
void shuffle(RandomIt first, RandomIt last, URBG&& g);
```

- *URBG* : Uniform Random Bit Generator (e.g., `default_random_engine`, `mt19937`)

Example

```
#include <random>
#include <algorithm>

int main() {
        vector<int> v = {1, 2, 3, 4, 5};

        // Initialise random number generator
        random_device rd;
        mt19937 g(rd());

        // Shuffle the vector using shuffle (modern C++)
        shuffle(v.begin(), v.end(), g)

        return 0;
}
```

Key Differences between `random_shuffle` and `shuffle`

- `shuffle` requires an explicit random number generator, making it more flexible and reliable for good randomness
- `random_shuffle` was tied to the poor-quality `rand()` function, which led to it's deprecation and removal

- next_permutation()

next_permutation()

In C++ STL, `next_permutation()` is a function that rearranges the elements in a range to the **next lexicographical permutation_** (or order) of those elements. If the current arrangement is the largest possible permutation, it rearranges the elements to the **smallest permutation** (sorted in ascending order).

Function Prototype

```
bool next_permutation(BidirectionalIt first, BidirectionalIt last);
```

- *first* : Iterator pointing to the first element of the range
- *last* : Iterator pointing to one past the last element of the range

Return value

- Returns `true` if the function was able to rearrange the elements into the next permutation
- Returns `false` if the function could not find a next permutation (i.e., if the current arrangement is the largest permutation possible)

Key Points about `next_permutation()`

1. **Lexicographical Order** : The function generates permutations in lexicographical (dictionary) order
2. **Efficient Generation** : It efficiently generates the next permutation without generating all permutations at once, making it useful for algorithms requiring incremental permutations
3. **Sorting** : If the range is sorted in descending order (the largest lexicographical permutation), `next_permutation()` will sort the range in ascending order, returning `false`

Examples

`next_permutation()` with Strings

```
#include <algorithm>
#include <string>
using namespace std;

int main() {
    string s = "abc";

    do {
        cout << s << endl;
    } while (next_permutation(s.begin(), s.end()));

    return 0;
}
```

Output

```
abc
acb
bac
```

```
bca
cab
cba
```

After the last permutation (`cba`), the function returns `false` because there are no more possible permutations.

Sorting (Largest Permutation)

```
#include <algorithm>
#include <vector>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3}; // Starting with the smallest permutation

    // Print all permutations until reaching the largest
    do {
        for (int x : v) {
            cout << x << " ";
        }
        cout << endl;
    } while (next_permutation(v.begin(), v.end()));

    // After the loop, v is now {3, 2, 1}, which is the largest
permutation
    cout << "After reaching largest permutation: ";
    for (int x : v) {
        cout << x << " ";
    }
    cout << endl;

    // Call next_permutation again
    next_permutation(v.begin(), v.end());

    // Print the rearranged vector
    cout << "Rearranged to smallest permutation: ";
    for (int x : v) {
        cout << x << " ";
    }
    cout << endl;

    return 0;
}
```

Output

```
1 2 3
1 3 2
2 1 3
2 3 1
```

```
3 1 2
3 2 1
After reaching largest permutation: 1 2 3
Rearranged to smallest permutation: 1 3 2
```

The `do-while` loop executes once before checking the condition. This means that if the last permutation is printed, the vector will be rearranged to the smallest permutation on the next call of `next_permutation()` inside the loop, leading to `123`, and then `132`.

Practical Applications

1. **Generating Permutations** : `next_permutation()` is widely used when you need to generate permutations in lexicographical order
2. **Solving Puzzles/Problems** : It can be used to solve combinatorial problems that require exploring all possible orders of elements
3. **Brute Force Solution** : Useful in brute force algorithms that need to try all possible arrangements of elements, such as solving permutation-based puzzles or optimisation problems

Self Implementation of `next_permutation()`

Problem Link : [31. Next Permutation]

Platform : #Platform-LeetCode

Difficulty : #Difficulty-Medium

Approach-1
Generate all possible permutations using Recursion, then sort them in increasing order & return the next index

```
class Solution {
public:
    void generatePermutations(vector<int> &nums, vector<vector<int>>
&permutations, int ind) {
        if(ind == nums.size()) {
            permutations.push_back(nums);
            return;
        }

        for(int i=ind; i<nums.size(); i++) {

            // Avoid duplicate permutations when elements repeats
            // For better understanding, dry run {1, 1, 2, 3}
            if(i == ind || (i != ind && nums[ind] != nums[i])) {
                swap(nums[ind], nums[i]);
                generatePermutations(nums, permutations, ind+1);
                swap(nums[ind], nums[i]);
            }
        }
    }
```

```
    void nextPermutation(vector<int>& nums) {
        vector<vector<int>> permutations;

        // Generate all permutations in sorted order
        generatePermutations(nums, permutations, 0);
        sort(permutations.begin(), permutations.end());

        for(auto it:permutations) {
            for(auto p:it) cout << p << " ";
            cout << endl;
        }

        // Linear Search to find where does 'nums' lie
        int n = permutations.size(), ind;
        for(int i=0; i<n; i++) {
            if(permutations[i] == nums) {
                ind = i;
                break;
            }
        }

        // Next index permutation will be answer
        // If next index doesn't exist, then 1st will be answer
        nums = permutations[(ind+1) % permutations.size()];
    }
};
```

TC : $O(n!) + O(n! * \log(n!)) + O(n!)$
SC : $O(n! * n)$

Approach-2
Observations,

1. Next elements of a dictionary tend to have longer prefix match

Algorithm

1. Try getting longest prefix match possible, by finding the break point (increasing 0 to i+1 & decreasing from i+1 to n-1)
2. Once you figure out break point, after the longest prefix try putting someone at i who is slightly greater than current guy
3. Fill the remaining portion i+1 to n-1 as small as possible, by filling in sorted order
   Eg : {2, 1, 5, 4, 3, 0, 0}
   `i` will be pointing to value 5
   {2, 3, , , , , _}
   {2, 3, 0, 0, 1, 4, 5}
4. {1, 2, 3, 4, 5} dip right before the last element, so we know dip highest could be found at (n-2)th index
5. {5, 4, 3, 2, 1} whole decreasing, so no dip (as it's the last permutation)

```
class Solution {
public:
    void nextPermutation(vector<int>& nums) {
        int n = nums.size();

        // Find the dipIndex
        int dipIndex = -1;
        for(int i=n-2; i>=0; i--) {
            if(nums[i] < nums[i+1]) {
                dipIndex = i;
                break;
            }
        }

        // If there's no dip, then it would be lexicographically the
biggest permutation, so return smallest one
        if(dipIndex == -1) {
            reverse(nums.begin(), nums.end());
            return;
        }

        // Find someone ahead which is slightly greater than element at
dipIndex
        // Since right portion is decreasing & we need smallest, so
start traversing from back
        for(int i=n-1; i>=0; i--) {
            if(nums[i] > nums[dipIndex]) {
                swap(nums[i], nums[dipIndex]);    // Swapping let's [i+1
... n-1] stay in decreasing order
                break;
            }
        }

        // Fill the remaining portion as small as possible, by filling
it in sorted order
        // Since that part is decreasing, so we just reverse it
        reverse(nums.begin() + dipIndex+1, nums.end());
    }
};
```

TC : $O(N)$
SC : $O(1)$ $\quad$ (Modifying the given array)

- lower_bound() and upper_bound()

lower_bound() and upper_bound()

In C++ STL, `lower_bound` and `upper_bound` are 2 functions used to find elements or positions in sorted sequences (like `arrays`, `vectors`, or other sorted containers). Both functions are part of the `<algorithm>` header and are commonly used with containers that support **random access iterators**.

lower_bound()

The `lower_bound()` function returns an iterator pointing to the **first element** in the range that is **not less than** the specified value (i.e., it is greater than or equal to the given value).

Syntax :

```
RandomIt lower_bound(RandomIt first, RandomIt last, const T& value);
```

- *first* : Iterator pointing to the start of the range
- *last* : Iterator pointing to the end of the range (the position one past the last element)
- *value* : The value to search for

Example :

```
int main() {
        vector<int> v = {1, 3, ,3, 7, 9};
        auto it = lower_bound(v.begin(), v.end(), 3);

        // Print index and value of the lower bound
        cout << "LB of 3 is at index: " << it - v.begin() << ", value: "
<< *it << endl;
        // Output: LB of 3 is at index: 1, value: 3

        return 0;
}
```

In this example, `lower_bound` returns the iterator to the 1st occurrence of 3, or the position where 3 would be inserted it it's not present.

Characteristics of `lower_bound` :

- *Time Complexity* : $O(logN)$, where N is number of elements between `first` and `last` (binary search)
- If the value is not present, it returns an iterator to the position where the value would be inserted to maintain the sorted order
- It's index gives you the number of elements less than the specified value
- If lower bound doesn't exist, then it returns `v.end()` or `arr + n`, which gives us index as n (i.e. size)

upper_bound()

The `lower_bound()` function returns an iterator pointing to the **first element** in the range that is **greater than** the specified value.

Syntax :

```
RandomIt upper_bound(RandomIt first, RandomIt last, const T& value);
```

- *first* : Iterator pointing to the start of the range
- *last* : Iterator pointing to the end of the range (the position one past the last element)
- *value* : The value to search for

Example :

```
int main() {
        vector<int> v = {1, 3, ,3, 7, 9};
        auto it = upper_bound(v.begin(), v.end(), 3);

        // Print index and value of the lower bound
        cout << "UB of 3 is at index: " << it - v.begin() << ", value: "
<< *it << endl;
        // Output: UB of 3 is at index: 1, value: 3

        return 0;
}
```

In this example, `upper_bound` returns the iterator to the 1st element greater than 3, which is 7.

Characteristics of `upper_bound` :

- *Time Complexity* : $O(logN)$, where N is number of elements between `first` and `last` (binary search)
- If the value is not present, it returns an iterator to the position where a larger value would occur in the sorted order
- It's index gives you the number of elements less than or equal to the specified value
- If upper bound doesn't exist, then it returns `v.end()` or `arr + n`, which gives us index as n (i.e. size)

Key Differences between `lower_bound()` and `upper_bound()`

- `lower_bound` returns the iterator to the 1st element that is **greater than or equal** to the value
- `upper_bound` returns the iterator to the 1st element that is **strictly greater** than the value

NOTE : Both functions assume that the range `[first, last)` is sorted

- reverse()

reverse()

In C++ STL, the `reverse()` function is used to reverse the order of elements in a given range. This function is part of the `<algorithm>` header and works with any container that supports bidirectional or random access iterators (such as `vector`, `deque`, `array`, etc.)

Function Prototype

```
void reverse(BidirectionalIt first, BiDirectionalIt last);
```

- *first* : Iterator pointing to the beginning of the range
- *last* : Iterator pointing to the end of the range (the position one past the last element)

Time Complexity : $O(N)$, where N is the number of elements in range `[first, last)`.

Example

```
#include <algorithm>

int main() {
        vector<int> v = {1, 2, 3, 4, 5}

        // Reversing the entire vector
        reverse(v.begin(), v.end());

        // Reversing a part of the vector (from index 1 to 4)
        reverse(v.begin() + 1, v.begin() + 5);

        return 0;
}
```

How `reverse()` works ?

- The function works by swapping elements starting from both ends of the range and moving towards the center.
- It requires the container to have bidirectional iterators, which allow moving forward and backward in the range (containers like `list`, `vector`, `deque` support this).

- sort() and Lambda Comparators

sort()

The `sort()` function in C++ STL is a powerful algorithm used to sort elements in a range (such as arrays or containers like `vector`, `deque`, etc.). It is a part of the `<algorithm>` header, and it's highly efficient, typically implemented using the **Introsort** algorithm, which is a hybrid sorting algorithm combining Quick Sort, Heap Sort, and Insertion Sort (because these 3 don't require extra spaces).

Function Prototype

```
void sort(RandomIt first, RandomIt last);
```

- *first* : Iterator pointing to the beginning of the range
- *last* : Iterator pointing to the end of the range (the position one past the last element)

Key Points about `sort()`

1. **Random Access Iterators** : `sort()` requires the iterators to be random access (e.g., for `vector`, `array`, `deque`). Containers like `list` and `forward_list` that do not provide random access iterators cannot use `sort()`.
2. **Default Sorting (Ascending order)** : By default, `sort()` sorts elements in ascending order based on the `<` operator.
   - For integers, strings, floating-point numbers, etc., it uses the built-in comparison operators.
   - For custom data types, you need to overload the `<` operator or provide a custom comparator.
3. **Time Complexity** : Average-case and worst-case time complexity is **O(NlogN)**, where **N** is the number of elements to be sorted.
4. **Custom Comparators** : You can provide a custom comparator (a function or a function object) to define the sorting criterion. This is useful when you want to sort in descending order or use custom rules for sorting.

Examples of using `sort()`

Sorting in Ascending Order

```
sort(v.begin(), v.end());    // Sorting an vector
sort(v, v + n);    // Sorting an array
```

Partial Sorting (using `sort()` on a Range)

```
// Sort elements from idnex 1 to 4 (excluding index 4)
sort(v.begin() + 1, v.begin() + 4);
```

Sorting with Custom Comparator (User-defined Objects)

```
struct Person {
        string name;
        int age;
}

// Custom comparator to sort by age
bool compareByAge(const Person &a, const Person &b) {
        return a.age < b.age;    // Sort in ascending order of age
}

int main() {
        vector<Person> people = {{"Kartik", 21}, {"Cat", 19}, {"Mayank",
31}, {"Mohit", 18}};

        // Sort people by age
        sort(people.begin(), people.end(), compareByAge);

        return 0;
}
```

Lambda Expressions

In C++11 and Later, you can create a comparator **inline** using **lambda expressions**. This makes it easier to write short, simple comparators without needing to define a separate function. Lambda expressions provide a concise way to create anonymous functions directly at the place where you need them.

Syntax of Lambda Expressions

A Lambda expression has the following general form:

```
[ capture ] ( parameters ) -> return_type {body}
```

- **Capture** : Specifies which variables from thee surrounding scope should be accessible in the lambda.
  Common captures include:

- `[ ]` (no capture)
- `[=]` (capture all variables by value)
- `[&]` (capture all variables by reference)
- `[x]` (capture only `x` by value), `[&x]` (capture only `x` by reference)

- **Parameters** : The function arguments (optional)
- **Return Type** : Can be inferred (optional)
- **Body** : The function body (code to execute)

Examples of using Lambda Expressions

Sorting Custom Objects with a Lambda Comparator

```
struct Person {
        string name;
        int age;
}

int main() {
        vector<Person> people = {{"Kartik", 21}, {"Cat", 19}, {"Mayank",
31}, {"Mohit", 18}};

        // Sort people by age using a lambda expression
        sort(people.begin(), people.end(), [](const Person &a, const
Person &b){
                return a.age < b.age;    // Sort in ascending order of
age
        });

        return 0;
}
```

Sorting by Multiple Criteria using a Lambda

```
struct Person {
        string name;
        int age;
}

int main() {
        vector<Person> people = {{"Kartik", 21}, {"Cat", 19}, {"Mayank",
31}, {"Mohit", 18}};

        // Sort people by age using a lambda expression
        sort(people.begin(), people.end(), [](const Person &a, const
Person &b){
                if(a.age == b.age) {
                        return a.name < b.name;    // If ages are equal,
sort by name
                }
                return a.age < b.age;    // Sort in ascending order of
age
        });
```

```
        return 0;
}
```

Advantages of using Lambda Functions for Comparators

1. **Conciseness** : Inline lambda functions eliminate the need to write separate comparator functions, making your code more compact and readable.
2. **Ease of Use** : For simple comparators, lambdas are quick to write and don't require additional function definitions.
3. **Clarity** : The sorting logic is placed where it is used, improving the code's readability by keeping relevant parts close together.

- __gcd() and gcd()

__gcd()

In C++ STL, `__gcd()` is a function used to compute the **Greatest Common Divisor (GCD)** of 2 integers. It is a part of the `<algorithm>` header.

Function Prototype

```
int __gcd(int a, int b);
```

- *Parameters* : Two integers `a` and `b`
- *Returns* : The largest positive integer that divides both `a` and `b`

It uses **Euclidean algorithm** under the hood, which is very efficient for calculating GCD.

NOTE
However, from C++17 onwards, the function `gcd()` is officially introduced, which should be preferred over `__gcd()` for better standard compliance.

gcd()

`gcd()` is the official GCD function introduced in C++17, part of the `<numeric>` header.

Function Prototype

```
template <class T>
T gcd(T a, T b);
```

- *Template Type* : `T` can be any integral type (`int` , `long` , `long long` , etc.)
- *Parameters* : Two integers `a` and `b` of the same type
- *Returns* : The greatest common divisor of `a` and `b` , as a value of type `T`

We should prefer `gcd()` for modern C++ code because it's standardised and works for different integral types.

- min_element() and max_element()

min_element() and max_element()

In C++ STL, `min_element` and `max_element` are functions that are used to find the **minimum** and **maximum elements**, respectively, in a given range of elements (like arrays,

vectors, or lists). Both functions are part of the `<algorithm>` header.

Key Points

- Both functions work with any container that provides forward iterators, such as `vector`, `list`, or `arrays`
- They both operate in $O(N)$ time complexity
- Custom comparators can be used to define how elements should be compared

min_element()

Function Prototype

```
template<class ForwardIt> ForwardIt min_element(ForwardIt first,
ForwardIt last);
```

- *first* : An iterator pointing to the first element of the range
- *last* : An iterator pointing to one past the last element of the range

Return Value

- Returns an iterator pointing to the smallest element in the specified range
- If the range is empty, it returns `last`

max_element()

Function Prototype

```
`template<class ForwardIt> ForwardIt max_element(ForwardIt first,
ForwardIt last);`
```

- *first* : An iterator pointing to the first element of the range
- *last* : An iterator pointing to one past the last element of the range

Return Value

- Returns an iterator pointing to the largest element in the specified range
- If the range is empty, it returns `last`

Example with Custom Comparators

Both functions can also accept a custom comparator to determine how to compare the elements

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>
using namespace std;

int main() {
    vector<int> numbers = {-5, 3, -8, 1, 4};
```

```
    // Finding the minimum element based on absolute values
    auto min_it = min_element(numbers.begin(), numbers.end(), [](int a, int b) {
            return abs(a) < abs(b);
        });

    if (min_it != numbers.end()) {
        cout << "The minimum element based on absolute value is: " << *min_it << endl;
    }

    // Finding the maximum element based on absolute values
    auto max_it = max_element(numbers.begin(), numbers.end(), [](int a, int b) {
            return abs(a) < abs(b);
        });

    if (max_it != numbers.end()) {
        cout << "The maximum element based on absolute value is: " << *max_it << endl;
    }

    return 0;
}
```

Output

```
The minimum element based on absolute value is: 1
The maximum element based on absolute value is: -8
```

For reference, use website [cplusplus]

2. Containers
   These are data structures like arrays, vectors, linked lists etc. that store collections of objects.